

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Case Study

COURSE NAME: Cloud Computing and
Big Data Analytics

COURSE CODE: CSA15

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)*

Assessment Tool Weightage: 20%

Dave's Taxonomy: Precision (Level 3)
Question Count: 3

Total Marks: 30

Code of Conduct

I **K.MAHESH BABU(192525122)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **K.MAHESH BABU(192525122)**

Assessment Tasks

Case Study

1. Case 1: A food delivery company wants to deploy a cloud-native backend with web APIs, browser clients, and shared cloud storage for receipts and delivery images. Propose the architecture and explain the role of each component.
2. Case 2: A media startup must decide between SAN-backed storage, object storage, and serverless processing for video upload workflows. Analyze the case and recommend a deployment model.
3. Case 3: A smart city platform needs low-latency decision making near sensors while still using the cloud for analytics. Analyze the use of edge computing and fog computing in this environment.

Case Study Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Case	25%	Clearly identifies all technical and business requirements	Identifies most requirements	Basic understanding only	Poor understanding of case
Application of Concepts	25%	Applies suitable cloud concepts and tools effectively	Good application with minor issues	Partial application	Weak or incorrect application
Analysis	20%	Strong analysis with clear trade-off reasoning	Reasonable analysis	Limited analysis	Minimal analysis
Solution Design	20%	Solution is feasible, practical, and well justified	Reasonable solution with minor gaps	Basic solution	Unclear solution
Clarity	10%	Well-structured and easy to follow	Mostly clear	Somewhat unclear	Poorly organized

Case 1: Cloud-native backend for a food delivery company

Understanding of the case:

The company needs a cloud-native backend, web APIs for communication between clients and services, browser-based access, and shared cloud storage for receipts and delivery images. The design should be scalable, secure, reliable, and easy to maintain.

Proposed architecture:

Browser/mobile clients → HTTPS/REST Web API → API gateway/load balancer → application services → managed database. Receipts and delivery images are stored in object storage. Authentication and authorization protect the APIs. Monitoring and centralized logging are used to observe application health and failures.

Role of each component:

1. Browser/mobile client: Provides the user interface for customers, delivery staff, and administrators.
2. REST Web API: Receives requests such as placing orders, tracking deliveries, and uploading receipts.
3. API gateway/load balancer: Routes requests, supports security controls, and distributes traffic across backend instances.
4. Cloud application services: Execute business logic such as order management, payment status, and delivery tracking.
5. Managed database: Stores structured data such as users, orders, restaurants, and delivery status.
6. Object storage: Stores unstructured files such as receipts and delivery images; it is suitable for large files and scalable storage.
7. Identity and access management: Controls who can access APIs, databases, and stored objects.
8. Monitoring and logging: Helps detect errors, performance problems, and security events.

Analysis and justification:

Object storage is preferable for receipts and images because these are file objects rather than database records. REST APIs provide a simple and widely used interface between clients and cloud services. A managed database reduces infrastructure administration, while scalable application services allow the backend to handle changes in demand.

Final solution:

Use a cloud-native, API-driven architecture with HTTPS/REST APIs, managed application services, a managed database, object storage for receipts/images, IAM for access control, and monitoring/logging. This solution is practical, scalable, secure, and suitable for real-world food delivery workloads.

Case 2: Storage and processing model for a video upload workflow

Understanding of the case:

The media startup must handle video uploads and choose among SAN-backed storage, object storage, and serverless processing. Video files are large unstructured objects and the workflow may experience changing upload and processing volumes.

Comparison:

SAN-backed storage provides high-performance block-level storage and is useful when applications need block devices and predictable storage performance. Object storage is designed for large unstructured files such as videos and can scale to very large data volumes. Serverless processing executes event-driven functions without requiring the startup to manage servers directly.

Recommended deployment model:

Use object storage as the primary video repository and serverless processing for automated workflow steps. When a video is uploaded, an event can trigger serverless processing to perform tasks such as validation, metadata extraction, thumbnail generation, or starting transcoding workflows.

Trade-off analysis:

Object storage is more suitable than SAN-backed storage for the original video library because videos are large objects and need scalable capacity. Serverless processing reduces infrastructure management and can scale according to events. SAN/block storage should be introduced only for processing components that specifically require high-performance block-level storage.

Final solution:

Recommended model: Object storage + event-driven serverless processing, with optional high-performance block storage for specialized processing workloads. This gives the startup scalability, lower operational effort, and a practical pay-for-use processing architecture.

Case 3: Edge and fog computing for a smart city**Understanding of the case:**

The smart city platform needs low-latency decisions close to sensors while still using cloud computing for large-scale analytics. Examples include traffic signals, surveillance alerts, environmental sensors, and emergency notifications.

Role of edge computing:

Edge computing places computation close to the sensors or devices. Immediate processing can identify events and produce fast decisions without sending every raw data item to a distant cloud. This reduces latency and can reduce the amount of data transferred to the cloud.

Role of fog computing:

Fog computing provides an intermediate computing layer between edge devices and the cloud. Local gateways or regional nodes can aggregate sensor data, perform additional processing, and coordinate multiple edge devices before forwarding selected information to the cloud.

Recommended architecture:

Sensors/cameras → Edge devices → Fog gateway/regional node → Cloud platform → Big-data analytics/dashboard. Edge devices handle urgent decisions, fog nodes aggregate and coordinate local data, and the cloud performs long-term storage, large-scale analytics, model training, and city-wide reporting.

Analysis and justification:

Using only the cloud may introduce network latency for time-critical decisions. Using only edge devices can make city-wide analytics and centralized management difficult. A combined edge-fog-cloud architecture balances latency, bandwidth, scalability, and centralized analytics.

Final solution:

Use edge computing for immediate sensor-level decisions, fog computing for local aggregation and coordination, and cloud computing for centralized storage, big-data analytics, reporting, and long-term intelligence. This is a practical architecture for smart-city applications requiring both fast response and large-scale analysis.